

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Краткое руководство по системе MATLAB
с приложениями

Методические указания и контрольные задания

Санкт-Петербург
2000

Утверждено на заседании
кафедры общей математики и информатики
в качестве методических указаний
для студентов естественных факультетов

Составители: А.Е.Барабанов, А.К.Пономаренко

СОДЕРЖАНИЕ

Краткое руководство по языку MATLAB	1
1. Окно и интерпретатор	1
2. Числа и матрицы	1
3. Выражения	4
4. Основные операторы	5
5. Встроенные функции	7
6. Графики	8
7. m-файлы и функции	9
8. Запись и считывание с диска	11
9. Библиотеки стандартных функций	11
Стандартные процедуры интегрирования в пакете FUNFUN на языке MATLAB	12
Интерполирование функций	14
Метод Рунге-Кутты решения задачи Коши для обыкновенных дифференциальных уравнений	19
Литература	26

Первые шаги программиста, осваивающего новый пакет программ, обычно сопровождаются обилием простейших вопросов, без решения которых просто невозможно выполнить работу. Ответы на многие из них собраны в данном руководстве, которое ни в какой степени не претендует на полноту изложения, характерного для учебников. Собственно учебник (на английском языке) является составной частью системы MATLAB и вызывается нажатием кнопки Help в главном меню.

1. Окно и интерпретатор

Система MATLAB предназначена для численных расчетов и просмотра результатов на графиках. После запуска программы MATLAB на экране появляется рабочее окно, в котором можно набирать команды алгоритмического языка MATLAB. Команды набираются в строке, начинающейся приглашением "»", в которой находится мигающий курсор, указывающий позицию для очередного вводимого символа.

В отличие от большинства наиболее распространенных языков программирования MATLAB является интерпретатором, т.е. для выполнения программы от программиста не требуется специально транслировать полный текст программы. Команда, набранная в рабочей строке, выполняется сразу же после нажатия клавиши ввода, завершающей эту строку, а результаты работы команды остаются в оперативной памяти и могут быть использованы в следующих командах. Таким образом, после выполнения программы все промежуточные и окончательные результаты счета не исчезают, как это происходит в системах FORTRAN, PASCAL, C, ALGOL и пр., а остаются в распоряжении программиста.

2. Числа и матрицы

В большинстве языков программирования числа могут храниться в памяти в разных видах: как целое с 8, 16 или 32 двоичными разрядами, как вещественное число с фиксированной или плавающей запятой, как элемент массива. Поэтому программист должен специально указывать тип числа, для чего служит описание переменных в начале программы. В языке MATLAB никаких описаний нет, и программист может использовать произвольные идентификаторы, не заботясь об их описании. По определению, любой идентификатор обозначает матрицу, состоящую из комплексных чисел с плавающей запятой. Исключение составляет лишь строка символов, которая начинается и заканчивается одинарными кавычками.

Размер произвольной матрицы a есть пара $[m, n]$, где m – количество строк, а n – количество столбцов. Если в рабочей строке

набрать `size(a)` и ввод, то на экране в качестве ответа системы появятся эти два числа m и n , являющиеся результатом выполнения стандартной встроенной функции `size`. Обычное число есть по определению матрица размера $[1, 1]$.

Хотя каждое число (в том числе и переменные цикла i , j и т.д.) записывается в памяти как комплексное с плавающей запятой, программист может использовать привычные обозначения: для вещественных чисел мнимая часть просто не указывается. Так, записи 10 и 10.00 обозначают одно и то же вещественное число, у которого мнимая часть считается равной нулю, так как не указана. Для записи мнимой части нужно подставить символ i или j непосредственно после числа без пробелов. Например, $3.1 - 2.0i$ есть комплексное число с вещественной частью 3.1 и мнимой частью -2 .

Для ввода матрицы поэлементно используются квадратные скобки. При этом элементы вводятся построчно, внутри строки они разделяются пробелом или запятой, а окончание строки обозначается точкой с запятой. Например, команда

$$a = [1 \quad -2 \quad 3 \quad -4; \quad -5, \quad 6, \quad -7 \quad 8; \quad 9 \quad -10 \quad 11, \quad -12]$$

определяет матрицу

$$a = \begin{pmatrix} 1 & -2 & 3 & -4 \\ -5 & 6 & -7 & 8 \\ 9 & -10 & 11 & -12 \end{pmatrix}.$$

Матрицы с одной строкой и равноотстоящими элементами записываются при помощи двоеточия. Например, $[-2 : 10]$ есть матрица, имеющая только одну строку, в которой стоят числа $-2, -1, 0, 1, \dots, 10$, всего 13 элементов. Здесь по умолчанию считается, что шаг между соседними элементами равен 1. Однако, его можно указать явно, например, $[1 : 0.01 : 2]$ есть строка с элементами 1, 1.01, 1.02, ..., 1.99, 2, всего 101 элемент. Таким способом удобно, в частности, задавать сетку отсчетов по оси абсцисс при построении графиков.

Элемент матрицы записывается в круглых скобках. В примере $a(2, 3)$ есть число -7 . Если матрица b имеет только одну строку, то обращение $b(2, 1)$ есть ошибка, а обращение $b(1, 2)$ выбирает второй элемент строки. Если матрица c имеет только один столбец, то обращение $c(1, 2)$ есть ошибка, а $c(2, 1)$ выбирает

второй элемент столбца. Однако, обращения $b(2)$ и $c(2)$ возможны и будут восприняты правильно, так как оба массива b и c — одномерные.

Все матрицы не имеют фиксированных границ для своих размеров (кроме емкости всей оперативной памяти) и могут наращиваться обычным оператором присваивания. Например, матрица a имеет размер $[3, 4]$. Но если ввести команду $a(2, 6) = 15$, то получится матрица

$$a = \begin{pmatrix} 1 & -2 & 3 & -4 & 0 & 0 \\ -5 & 6 & -7 & 8 & 0 & 15 \\ 9 & -10 & 11 & -12 & 0 & 0 \end{pmatrix}.$$

Ее размер стал $[3, 6]$, а новые незадаанные элементы заполнились нулями.

Из матрицы можно вырезать любую подматрицу, расположенную на указанных строках и столбцах. Например, матрица $a0 = a([1, 3], [3, 4, 6])$ содержит первую и третью строки матрицы a , на которых остаются элементы с номерами столбцов 3, 5 и 6:

$$a0 = \begin{pmatrix} 3 & -4 & 0 \\ 11 & -12 & 0 \end{pmatrix}.$$

При назначении индексов вырезки можно пользоваться двоеточием для перечисления всех индексов с шагом 1. Например, $a1 = a(1 : 2, 1 : 3)$, $a2 = a(3, 1 : 3)$, $a3 = a(:, 4 : 6)$. Двоеточие в первом аргументе выражения для матрицы $a3$ означает, что выбираются все имеющиеся индексы.

Матрицы можно составлять не только явно из чисел, но и из матриц, располагая их рядом или одну над другой. При этом надо позаботиться о том, чтобы размерности составляющих были согласованы так, чтобы получилась прямоугольная матрица. В нашем примере матрица $[[a1; a2], a3]$ совпадает с матрицей a . Выражение $[a1; a2]$ имеет смысл, так как под матрицей $a1$ размера $[2, 3]$ можно подписать матрицу $a2$ размера $[1, 3]$, и тогда получится прямоугольная матрица размера $[3, 3]$. Справа от нее можно добавить матрицу $a3$, так как количество строк у них одинаково.

В прикладных задачах часто приходится использовать матрицы, состоящие из одних нулей или из одних единиц, или единичные матрицы разных размерностей. Для них существуют стандартные встроенные функции. Функция `zeros(m, n)` задает матрицу размера $[m, n]$ из одних нулей; функция `ones(m, n)` — матрицу

размера $[m, n]$ из одних единиц. Обозначение $\text{zeros}(n)$ равнозначно $\text{zeros}(n, n)$, а $\text{ones}(n)$ равнозначно $\text{ones}(n, n)$. Единичная матрица размерности n (содержащая все нули кроме главной диагонали, где стоят единицы) обозначается $\text{eye}(n)$.

При работе с матрицами часто используются функции size и length . Результатом работы функции $\text{size}(a)$ является строка $[m, n]$, где m – количество строк матрицы a , а n – количество ее столбцов. $\text{length}(a)$ есть наибольшее из чисел m и n . Транспонирование матрицы обозначается штрихом: a' есть матрица a транспонированная.

3. Выражения

Матрицы можно складывать и умножать по обычным правилам алгебры. При умножении двух матриц количество столбцов левой матрицы должно быть равно количеству строк правой матрицы. Исключение составляют матрицы размера $[1, 1]$, т.е. числа. Их можно умножать на любую матрицу слева и справа. Числа можно также делить. Обозначения для арифметических действий стандартные: $+$, $-$, $*$, $/$. Кроме того, возведение в степень обозначается символом $\wedge$.

Чрезвычайно удобной особенностью языка MATLAB является возможность записи обратной матрицы при помощи обычного деления. Как известно, система линейных уравнений $Ax = b$ с квадратной невырожденной матрицей A имеет решение $x = A^{-1}b$. При заданных матрицах A и b решение системы линейных уравнений записывается одной командой: $x = A \backslash b$, где символ $\backslash$ означает деление матрицы b на матрицу A слева, т.е. умножение слева на A^{-1} . Выражение XY^{-1} с квадратной невырожденной матрицей Y записывается X/Y . Таким образом, выражение $/Y$ означает умножение на Y^{-1} справа. Если Y – число, то это просто деление на Y .

Если две матрицы имеют одинаковые размерности, то можно определить арифметические операции покомпонентно. Эти операции осуществляются над парами чисел, расположенных на одинаковых местах в матрицах. Покомпонентные операции обозначаются обычными символами, перед которыми ставится точка. Это операции: $+. - . * ./ .^$. Например, если

$$X = \begin{pmatrix} 0 & 1 & 2 \\ -1 & -2 & -3 \end{pmatrix}, \quad Y = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix},$$

то

$$X.*Y = \begin{pmatrix} 0 & 2 & 6 \\ -4 & -10 & -18 \end{pmatrix}.$$

4. Основные операторы

Операторы присваивания, условные операторы и операторы цикла составляют основную часть прикладных программ. Оператор присваивания имеет синтаксис

Имя_переменной = Выражение

где в левой части стоит идентификатор или вырезка из матрицы, а в правой части – либо числовое выражение, либо строка символов, заключенная в простые кавычки, например, 'xyz'. Если в левой части не указаны индексы, то в оперативной памяти заводится новая переменная с указанным именем, которая принимает значение из выражения в правой части. Если в памяти уже была переменная с таким именем, то эта старая переменная предварительно стирается. Если в левой части указаны индексы, например, $a(1 : 2, 4 : 6)$, то переменная a , если она существовала, не стирается, а выражение из правой части подставляется в подматрицу, указанную индексами. Разумеется, размерность правой части в этом случае должна в точности совпадать с размерностью левой (в примере это матрица с двумя строками и тремя столбцами).

Если после оператора присваивания написана точка с запятой, то результат вычисления не отображается на экране, а в отсутствии точки с запятой – отображается. В длинной программе, написанной без знаков ; после операторов присваивания, результат выполнения каждого оператора выводится на экран, что засоряет интересующую программиста информацию. Поэтому рекомендуется ставить знаки ; после всех операторов, которые не заканчиваются словом end.

Условный оператор начинается словом if и заканчивается словом end. Чаще всего он записывается в виде

```
if      Условие
      Операторы
end
```


Здесь условие является логической формулой, например, $a < b$ или $((a \geq b) \& (c == d)) | (e \sim = f)$, где a, b, c, d, e, f – выражения. Знак $\geq$ означает "больше или равно", знак $==$ означает равенство двух выражений, знак $\sim =$ означает неравенство; символ $\&$ есть логическое "и"; символ $|$ есть логическое "или". В целом, условие – это то, что после вычислений может оказаться или истинным, и тогда следующие за ним операторы будут выполняться, или ложным, и тогда условный оператор заканчивает свою работу.

Условные операторы могут содержать также инструкции "else" и "elseif". В общем виде условный оператор имеет синтаксис

```

if      Условие1
        Операторы1
elseif  Условие2
        Операторы2
...
elseif  Условие-n
        Операторы-n
else
        Операторы0
end

```

Здесь только первая и последняя строчки обязательные, а остальные могут отсутствовать. Строчек elseif может быть сколько угодно, строчек со словом else – не более одной. Условия просматриваются по очереди, и как только одно из них оказывается истинным, выполняются следующие за ним операторы. Если все они ложные, то выполняются Операторы0, стоящие после слова else.

Операторы цикла бывают двух видов: со словом for и со словом while. Первый из них имеет синтаксис

```

for  Name = StartValue : Different : EndValue
    Тело цикла
end

```

Последовательность операторов в теле цикла выполняется для каждого значения переменной *Name*, начиная со *StartValue* с шагом *Different* до значения *EndValue*. Средний аргумент *Different* может не указываться, и тогда он считается равным 1. Весьма

распространенное начало оператора цикла имеет вид "for $i = 1 : n$ ".

Второй оператор цикла имеет синтаксис

```

while  Условие
    Тело цикла
end

```

Тело цикла выполняется, если условие истинно. После выполнения тела цикла условие проверяется снова и т.д. Операторы такого вида вполне могут привести к заикливанию программы, в частности, если результаты выполнения тела цикла никак не сказываются на условии, которое остается всегда истинным.

Операторы цикла обоих видов немедленно прекращают свою работу, если в теле цикла встречается команда "break". В случае нескольких вложенных операторов цикла прекращается работа только самого внутреннего из них.

5. Встроенные функции

Стандартные математические функции:

sin, cos, log, exp, tan, asin, acos, atan

являются встроенными в системе MATLAB. Если x – число, то выражение $\sin(x)$ вычисляет его синус. Аргументы функций записываются в круглых скобках.

Кроме указанных, встроенными являются многие другие важные математические функции, из которых выделим следующие:

abs(x) – абсолютная величина вещественного числа x
или модуль комплексного числа x ,
sqrt(x) – квадратный корень,
round(x) – округление до ближайшего целого,
real(x) – вещественная часть комплексного числа x ,
imag(x) – мнимая часть комплексного числа x ,
angle(x) – вычисление аргумента комплексного числа x ,
rand – генерация случайных чисел и матриц.

В качестве аргумента x каждой из перечисленных функций может быть подставлена матрица произвольного размера. Тогда эти функции вычисляются отдельно для каждого элемента матрицы, а результатом является матрица, составленная из вы-

численных значений и имеющая ту же размерность, что и исходная матрица x . Например, для вычисления функции $\log(x)$ на промежутке $2 \leq x \leq 5$ с шагом 0.01 достаточно выполнить команду $y = \log([2 : 0.01 : 5])$, и найденные 301 число составят строку y размера $[1, 301]$.

6. Графики

В последних версиях системы MATLAB имеются разнообразные средства отображения числовой информации на двумерных и трехмерных изображениях. Наиболее важной и простой из них является функция `plot`, рисующая один или несколько плоских графиков.

Пусть функция $y = f(x)$ задана в конечном числе точек: $y_i = f(x_i)$, $1 \leq i \leq n$. Другими словами, заданы два одномерных массива x и y одинаковой длины n . Команда `plot(x, y)` выводит на экран картинку с графиком функции f , построенным по имеющимся точкам. Для этого на координатной плоскости наносятся точки $(x_i, y_i)_{i=1}^n$ и точки с соседними абсциссами соединяются отрезком. Получается ломаная линия, которая выглядит гладкой, если точек достаточно много и они плотно расположены.

Если y — матрица размера $[m, n]$, а x — строка длины n , то команда `plot(x, y)` выводит на экран в одной картинке m графиков функций, по одному от каждой строки матрицы y . Эти графики будут разных цветов. Аргумент x может вообще отсутствовать, и тогда он заменяется на индексы массива y . Простейшим способом увидеть на графике массив y является команда `plot(y)`, и тогда недостающий массив x заменяется на $[1 : m]$, где m — количество строк y .

При выводе на экран нескольких кривых на одной координатной плоскости бывает удобно, чтобы они визуально легко различались. Например, одна кривая наносится пунктиром, другая сплошной линией, третья кружочками и т.д. Для этой цели предназначен третий аргумент функции `plot`. Команда `plot(x, y, 'инструкция')` формирует кривую так, как указано в инструкции, вокруг которой должны быть простые кавычки. Инструкция состоит из двух частей, обычно занимающих по одному символу и располагающихся в произвольном порядке. Первая часть указывает способ нанесения кривой, например: `'—'` — сплошная линия, `'.'` — точечная линия, `'—.'` — прерывистая линия, `'-.'` — штрихпунктирная линия и т.д. Полный набор вариантов можно про-

читать, введя команду `Help plot`. Вторая часть указывает цвет кривой одним символом, например: `'b'` — синий, `'w'` — белый, `'g'` — зеленый, `'r'` — красный и т.д. Полная инструкция для синей сплошной линии записывается `'b—'`. Указанные тройки $x, y, 's'$ могут повторяться внутри одной команды:

`plot(x1, y1, 's1', x2, y2, 's2', ..., xn, yn, 'sn').`

Каждая картинка с графиками одной или нескольких функций составляет отдельное Windows-приложение. Эти приложения нумеруются и называются Figure No. 1, Figure No. 2, ... Если открыты несколько таких приложений и появляется новая команда `plot`, то очередной график будет изображен на приложении, которое было активным последним по времени. Для открытия нового приложения Figure No. m , а также для просмотра уже открытого приложения Figure No. m , где m — натуральное число, применяется команда `Figure(m)`.

Например, одну волну синусоиды, нарисованную красным цветом на рисунке 2, можно увидеть, задав команду `Figure(2)` и затем команду `plot([0 : 0.01 : 2*pi], sin([0 : 0.01 : 2*pi]), 'r—')`. Заметим, что число $\pi = 3.14159...$ является стандартной константой языка MATLAB.

7. m-файлы и функции

Команды, введенные в строке рабочего окна, записываются в стековую память и могут воспроизводиться по одной нажатием клавиш $\downarrow$ и $\uparrow$. Естественно, длинную программу неудобно воспроизводить по одной команде, тем более, что память очищается после завершения сеанса работы. Поэтому предусмотрена возможность записи на жесткий диск программы на языке MATLAB. Файлы с программами имеют стандартное расширение `m`, и поэтому называются `m`-файлами (например, `test.m`).

Для запуска программы, содержащейся в файле `FileName.m`, достаточно выполнить команду `FileName` без указания расширения. При этом следует позаботиться, чтобы эта программа находилась в текущем каталоге либо в каталоге, указанном в списке путей. Если файл имеется в некотором каталоге, но система его "не видит", то она сообщает об ошибке, а именно, что `FileName` есть неопределенная переменная или функция. Рекомендуется записывать программы пользователя, относящиеся к одной те-

ме, в один рабочий каталог. Если система не находит нужную программу, то это говорит о том, что текущий каталог отличается от рабочего. Тогда надо воспользоваться командой MS DOS для смены текущего каталога: `cd путь`, где "путь" — это полный путь к каталогу, который нужно сделать текущим (например, `cd d:\users\ml\work`). Если в некоторой программе появляется команда `FileName` и если при этом в оперативной памяти нет переменных с таким именем, то система начинает искать файл `FileName.m` для того, чтобы вставить его содержимое полностью на место данной команды. Все переменные, определенные ранее в исходной программе, остаются доступными для вычислений в программе из файла `FileName.m` и наоборот, все новые переменные из этой программы остаются после ее завершения в оперативной памяти.

При подключении подпрограмм из `m`-файлов оперативная память этих программ и вызывающей программы объединяется, что не всегда удобно. Для разделения памяти разных подпрограмм применяются конструкции языка **MATLAB**, называемые функциями. Каждую функцию рекомендуется помещать в свой индивидуальный `m`-файл, причем имя файла должно совпадать с именем функции. Такой `m`-файл начинается строкой

```
function [result1,result2,...] = Fun_Name(parameter1, parameter2,...)
```

в которой `Fun_Name` — имя функции, `result1, result2, ...` — имена переменных, которые будут переданы в память вызывающей программы после окончания работы функции, а `parameter1, parameter2, ...` — это переменные, которые сообщает вызывающая программа для работы функции при помощи изготовления их дубликатов в памяти функции. Если некоторая переменная вызывающей программы должна возвратиться назад, то она должна быть как среди результатов `result1, ...,` так и среди параметров `parameter1, ...`. Обращение к данной функции в вызывающей программе имеет вид:

```
[result1,result2,...] = Fun_Name(parameter1, parameter2,...);
```

Переменные передаются также в память функции, если они объявлены как `global`. Например, команда `global x y z` объявляет переменные `x, y` и `z` глобальными, т.е. общими с головной программой и со всеми другими функциями, где они объявлены глобальными. Описание `global` должно предшествовать первому использованию таких переменных.

8. Запись и считывание с диска

Числовые данные можно записывать на диск и считывать с него. Простейший способ записи имеет синтаксис

```
save FileName variable1 variable2 ...
```

где `FileName` — имя файла; `variable1, variable2, ...` — переменные, которые записываются вместе с их идентификаторами. Файл, записанный таким способом, получает имя `FileName.mat` и информация в нем запакована таким способом, что ее невозможно прочитать обычными текстовыми редакторами. Для считывания данных из памяти применяется команда `load FileName` (с расширением `.mat` или без него), которая восстанавливает числовые значения переменных вместе с их идентификаторами.

Если требуется создать текстовый файл, доступный для чтения обычными редакторами, то каждую переменную рекомендуется записывать в свой индивидуальный файл командой `save FileName.Ext variable -ascii` или `save variable -ascii`. В первом варианте имя файла `FileName.Ext` указывается полностью, с (произвольным) расширением `.Ext`, а во втором варианте именем файла становится сам идентификатор переменной `variable` без расширения. В текстовых файлах имена переменных не записываются. Для чтения текстового файла применяется команда `load FileName.Ext -ascii`, причем прочитанный массив чисел обозначается идентификатором, совпадающим с именем файла `FileName`, от которого отрезано расширение `.Ext`, если оно было непустым.

9. Библиотека стандартных функций

Фирма **The MathWorks Inc.**, создатель системы **MATLAB**, выполнила большую работу для того, чтобы современные наиболее распространенные вычислительные алгоритмы были записаны в `m`-файлы на языке **MATLAB**. Получившаяся библиотека программ охватывает все области, обычно относящиеся к прикладной математике, а также разнообразные графические средства. Библиотека делится на каталоги, объединяющие программы, относящиеся к одному математическому разделу. Например, `elmat` — элементарные матричные преобразования, `datafun` — анализ данных и преобразование Фурье, `polyfun` — интерполяция и многочлены, `funfun` — интегрирование, дифференциальные уравнения и экстремумы функций и т.д.

Полный перечень каталогов и реализованных в них алгоритмов можно прочитать, нажав кнопку Help и затем Help Window в главном меню рабочего окна MATLAB. Каталоги имеют подкаталоги, к которым можно перейти, щелкнув мышкой на названии выбранного подкаталога. Каждый алгоритм реализован как функция в соответствующем m-файле. В начале каждого такого файла имеется краткий комментарий (на английском языке) о том, как пользоваться этой функцией и что она делает. Комментарии можно вызвать непосредственно в рабочее окно MATLAB командой help Function, где Function – имя функции. Например, команда help ode23 выводит на экран сообщение о том, как пользоваться функцией ode23, выполняющей решение системы обыкновенных дифференциальных уравнений.

Стандартные процедуры интегрирования в пакете FUNFUN на языке MATLAB

Функция $q = \text{quad}('fun', a, b)$ — интегрирование по адаптивной составной формуле парабол. Интегрирование производится по промежутку $[a, b]$, а интегрируемая функция f определяется подпрограммой, реализованной как функция языка MATLAB, обозначаемая строкой fun. Простая квадратурная формула парабол имеет вид

$$\int_a^b f(x) dx \approx \frac{b-a}{6} [f(a) + 4f(c) + f(b)], \quad \text{где } c = \frac{a+b}{2}.$$

Эта формула точна для всякой кубической функции f . Для некубических функций f погрешность приближения можно уменьшить, если разделить отрезок $[a, b]$ на n частей, применить формулу парабол к каждой из них и результаты сложить, т.е. если применить составную формулу парабол. Погрешность приближения не превосходит числа $M(b-a)^5/(180n^4)$, где M — максимум четвертой производной f на интервале (a, b) .

Число промежутков дробления n должно быть достаточно большим, чтобы обеспечить нужную точность вычислений и не слишком большим, чтобы увеличить скорость вычислений. В программе реализована адаптивная процедура дробления: каждый интервал делится пополам и вычисляются 3 числа: квадратурная сумма Q_0 для всего промежутка и квадратурные суммы

Q_1 и Q_2 для каждой половины. Фиксируются малые числа $\text{tol}(1)$ и $\text{tol}(2)$. Если выполнено неравенство

$$|Q_1 + Q_2 - Q_0| \leq \text{tol}(1)|Q_1 + Q_2| + \text{tol}(2),$$

то значение $Q = Q_1 + Q_2$ считается окончательным и дальнейшее деление интервала не производится. Если же это неравенство не выполнено, то каждый из половинных промежутков снова делится пополам и для него производится аналогичная операция сравнения.

Наиболее общий вид обращения к функции quad следующий:

$$[Q, cnt] = \text{quad}('fun', a, b, \text{tol}, \text{trace}, P1, P2, \dots).$$

Строка из двух элементов $\text{tol} = [\text{tol}(1), \text{tol}(2)]$ определяет относительную и абсолютную точность вычислений. По умолчанию $\text{tol}(1)=0.001$, $\text{tol}(2)=0$.

Параметр trace — отладочный. Если он присутствует и не равен нулю, то заставляет программу выводить на график текущие сведения о разбиении промежутка и вычислении функции fun. Переменные $P1, P2, \dots$ являются параметрами, участвующими в вычислении функции fun.

Результатом является число Q — приближенное значение интеграла, и количество cnt реальных обращений к вычислению подынтегральной функции fun.

Функция $q = \text{quad8}('fun', a, b)$ — интегрирование по адаптивной составной формуле Ньютона–Котеса 8 порядка. Отличается от предыдущей процедуры только видом квадратурной суммы на заданном промежутке. В формуле парабол промежуток интегрирования делится на две части и применяется формула Ньютона–Котеса второго порядка. В функции quad8 промежуток делится на 8 частей и применяется формула Ньютона–Котеса 8 порядка. Обычно это целесообразно при вычислении интегралов от негладких функций, т.е. имеющих большие производные или даже недифференцируемых.

Задание 1. Вычислить интеграл $\int_0^1 \sin(c_n x)/x dx$ при помощи функций quad и quad8. Здесь $c_n = 5 + 0.3n$, а n — номер варианта. Сравнить с точным значением интегрального синуса. Определить наибольший параметр $\text{tol}(1)$, при котором точность будет равна 10^{-5} , при условии $\text{tol}(2)=0$.

Интерполирование функций

Пусть в точках $x_0, x_1, \dots, x_n$, называемых узлами интерполяции, заданы значения функции f , равные $y_0, y_1, \dots, y_n$. Требуется построить алгоритм и написать программу для расчета значений функции f в любой другой точке x , отличной от узлов интерполяции.

Задача интерполирования не имеет смысла, если не предполагать дополнительных свойств функции между заданными узлами x_i . Обычные предположения состоят в том, что f — достаточно гладкая функция, т.е. имеет производные высоких порядков, которые ограничены известной величиной. Исторически первый способ интерполяции состоял в нахождении многочлена $P(x)$ степени не выше n , проходящего через заданные точки, т.е. для которого $P(x_i) = y_i$ при всех $0 \leq i \leq n$. Это интерполяционный многочлен Лагранжа, он может быть вычислен различными способами, например,

$$P(x) = \sum_{i=0}^n \frac{\omega(x)}{(x - x_i)\omega'(x_i)} y_i, \quad \text{где } \omega(x) = \prod_{j=0}^n (x - x_j).$$

Многие современные методы интерполяции основаны на понятии сплайна. Пусть набор узлов $X = (x_i)_{i=0}^n$ упорядочен: $a = x_0 < x_1 < \dots < x_{n-1} < x_n = b$, и пусть m и k — некоторые натуральные числа и $k < m$. Задан набор $Y = (y_i)_{i=0}^n$ значений функции в узлах из набора аргументов X .

Сплайном степени m и дефекта k , построенным по набору узлов X , называется любая функция f на промежутке $[a, b]$, удовлетворяющая следующим свойствам: 1) сужение f на каждый промежуток $[x_i, x_{i+1}]$ является многочленом степени не выше m ; и 2) функция f и все ее производные вплоть до порядка $n - k$ непрерывны в узлах интерполяции x_i , $0 \leq i \leq n$. Если кроме того, $f(x_i) = y_i$ при всех i , то такой сплайн называется интерполяционным. Интерполяционный сплайн, построенный по узлам X и значениям Y , обозначается $S_{X,Y}(x)$. Кубическим сплайном называется сплайн степени 3. Он может иметь дефект 1, 2 или 3, либо быть кубическим многочленом на всей оси (с дефектом 0).

Наибольшее распространение получили кубические сплайны дефекта 1. Они определяются однозначно по наборам X и Y , если заданы два дополнительных ограничения. Если такими ограни-

чениями являются $S''_{X,Y}(a) = 0$ и $S''_{X,Y}(b) = 0$, то сплайн называется вариационным, так как оптимизирует гладкость интерполированной функции по следующей теореме.

Теорема Холлидея о кубических сплайнах. Среди всех функций f , имеющих вторую производную на отрезке $[a, b]$ и удовлетворяющих интерполяционным ограничениям $f(x_i) = y_i$, $0 \leq i \leq n$, наименьшее значение функционала

$$J(f) = \int_a^b (f''(x))^2 dx$$

достигается на кубическом интерполяционном сплайне $S_{X,Y}(x)$ дефекта 1, удовлетворяющем ограничению $S''_{X,Y}(a) = S''_{X,Y}(b) = 0$.

Сплайн-интерполяция была реализована на языке MATLAB дважды: сначала в пакете POLYFUN — программы `spline`, `interp1`, `interp2`, `interp3`, а затем в специальном пакете программ SPLINES. Сплайны, являясь многочленами на каждом отрезке $[x_i, x_{i+1}]$, полностью определяются общей совокупностью коэффициентов этих многочленов. Поэтому рассчитать сплайн означает вычислить коэффициенты для каждого промежутка $[x_i, x_{i+1}]$, и эти коэффициенты записываются в память. Наиболее распространенным стандартом записи параметров сплайна является кусочно-полиномиальная (piece-wise polynomial) форма или pp-форма. В память записывается следующая строка: 10, d , n , X , k , M . Здесь 10 — код pp-формы (для сравнения: 11 и 12 — коды B-сплайна, 20 — код тензорного полиномиального сплайна и т.д.); d — размерность значений функции f ; n — число узлов минус 1; X — набор узлов; k — степень сплайна; M — матрица коэффициентов размера $dn \times k$, записанная по столбцам.

Пусть вектор X содержит узлы интерполяции, а вектор Y — значения функции в этих узлах. Команда $P = \text{spline}(X, Y)$ вычисляет вектор P , содержащий pp-форму кубического сплайна, построенного по паре X, Y и по двум дополнительным ограничениям: третьи производные непрерывны в предпоследних узлах с обоих концов. Значения полученного сплайна $\tilde{Y} = (\tilde{y}_j)_{j=1}^m$ в наборе аргументов $\tilde{X} = (\tilde{x}_j)_{j=1}^m$ вычисляются при помощи команды $\tilde{Y} = \text{ppval}(P, \tilde{X})$. Можно сразу вычислить значения $\tilde{y}_j$ сплайна в точках $\tilde{x}_j$ без записи в память pp-формы P . Для этого применяется команда $\tilde{Y} = \text{spline}(X, Y, \tilde{X})$.

Функция `interp1` осуществляет одномерную сплайн-интерполяцию различными способами. Команда $\tilde{Y} = \text{interp1}(X, Y, \tilde{X}, \text{'method'})$ определяет сплайн по исходным данным X, Y в соответствии с методом, указанным в строке `'method'` и затем вычисляет значения этого сплайна в аргументах, содержащихся в векторе $\tilde{X}$ и записывает результат в $\tilde{Y}$. Узлы интерполяции $X = (x_i)_{i=0}^n$ должны быть строго упорядочены: $a = x_0 < x_1 < \dots < x_{n-1} < x_n = b$. При многомерной интерполяции количество строк матрицы Y должно быть $n+1$. Строка `'method'` может принимать одно из следующих значений:

1. `'nearest'` — значение $\tilde{y}_j$ в точке $\tilde{x}_j$ равно значению y_i в точке x_i , ближайшей к точке $\tilde{x}_j$;
2. `'linear'` — линейная интерполяция (по умолчанию, т.е. при отсутствии строки `'method'` в обращении к функции `interp1`);
3. `'spline'` — кубические сплайны, обращение к функции `spline`;
4. `'cubic'` — локальная кубическая интерполяция.

Линейная интерполяция — это сплайны степени 1, графики которых состоят из отрезков, соединяющих на плоскости соседние интерполяционные точки (x_i, y_i) .

В локальной кубической интерполяции коэффициенты кубического полинома на промежутке $[x_i, x_{i+1}]$ полностью определяются значениями в четырех ближайших узлах: $y_{i-1}, y_i, y_{i+1}, y_{i+2}$. А именно, производные сплайна f в точках x_i и x_{i+1} определяются конечными разностями: $f'(x_i) = (y_{i+1} - y_{i-1})/2h$, $f'(x_{i+1}) = (y_{i+2} - y_i)/2h$, где шаг $h = x_{i+1} - x_i$ предполагается постоянным. По четырем числам $f(x_i), f'(x_i), f(x_{i+1}), f'(x_{i+1})$ однозначно находятся четыре коэффициента кубической функции. В итоге на промежутке $[a, b]$ получается кубический сплайн дефекта 2. Его особенностью является независимость значений от изменений исходных данных на расстоянии по оси абсцисс, большем, чем $2h$.

Функция `interp2` реализует двумерные сплайны, графики которых являются поверхностями в трехмерном пространстве. По конечному набору значений в узлах интерполяции определяется кусочно-полиномиальная функция с гладкой склейкой на границе полиномиальных участков. В качестве метода допускается только `'nearest'`, `'linear'` и `'cubic'`. Вариационные кубические сплайны в этой программе не реализованы. Аналогично, функция `interp3` реализует трехмерные сплайны, которые могут быть использованы для восстановления поля в трехмерном пространстве по заданным значениям в нескольких точках. Обращение к функциям

`interp2` и `interp3` отличается от обращения к функции `interp1` только количеством аргументов, задающих сетку узлов в многомерном пространстве.

Пакет программ **SPLINES** содержит значительно усовершенствованные методы построения и обработки полиномиальных сплайнов. Добавлены следующие группы функций: дифференцирование, интегрирование и арифметические операции над сплайнами; склейка и разделение сплайнов, а также вставка и удаление узлов интерполяции; обработка В-сплайнов; тензорные произведения сплайнов. В самих сплайнах допускается неравномерная гладкость склейки, что достигается введением кратных узлов интерполяции.

Функция `csape` строит pp-форму интерполяционного сплайна при различных условиях на концах промежутка и разными методами. Ее синтаксис: $P = \text{csape}(X, Y, \text{conds}, \text{valconds})$. Здесь X — набор узлов интерполяции, Y — матрица значений функции в этих узлах, `conds` — вид дополнительных условия, `valconds` — числовые данные к этим условиям. Граничные условия `conds` могут быть следующими:

1. `'complete'` — первые производные в крайних узлах совпадают с первыми производными кубической функции, построенной на крайних 4 узлах. Это условие выбирается по умолчанию, т.е. если аргументы `cond` и `valcond` отсутствуют в обращении к функции `csape`.
2. `'not-a-knot'` — в предпоследних узлах непрерывны все три производные, т.е. второй справа и второй слева узлы как бы отсутствуют (`knot` — по-английски узел).
3. `'periodic'` — первые и вторые производные в левом узле x_0 совпадают с соответствующими производными в правом узле x_n .
4. `'second'` — вторые производные в концевых узлах берутся из `valconds`.
5. `'variational'` — вторые производные в концевых узлах равны нулю.

Вместо указанных строковых значений `conds` может быть вектором 1×2 . В этом случае `conds(1)` указывает на номер производной, значение которой в левом концевом узле равно числу, записанному в `valconds(1)`, а `conds(2)` имеет аналогичный смысл для правого концевого узла. По умолчанию оба значения равны 1, т.е. задается первая производная. Например, если `conds=[2,2]`,

a valconds=[-1.1, 3.5], то заданы значения вторых производных $S''(x_0) = -1.1$, $S''(x_n) = -1.1$. Эти же два условия можно задать параметрами conds='second' и valconds=[-1.1, 3.5].

Функция csaps реализует аппроксимирующий сплайн. При заданных узлах интерполяции $X = (x_i)_{i=0}^n$ и значениях функции в них $Y = (y_i)_{i=0}^n$ аппроксимирующий сплайн $S(x)$ сохраняет гладкость, но необязательно проходит в точности через заданные узлы, $S(x_i) \neq y_i$. Это особенно важно в приложениях, где величины y_i являются результатами измерений в присутствии возмущений, которые имеют случайную природу и порождают ложные локальные экстремумы. Среди всех дважды дифференцируемых функций f минимум функционала

$$J_{\text{smooth}}(f) = (1 - \rho) \int_a^b (f''(x))^2 dx + \rho \sum_{i=0}^n |y_i - f(x_i)|^2$$

достигается на кубическом сплайне, который и называется сглаживающим. Параметр $\rho \in [0, 1]$ определяет баланс между стремлением провести функцию поближе к заданным точкам и стремлением повысить гладкость, уменьшив вторую производную. При $\rho = 1$ получается вариационный сплайн из теоремы Холлидея, а при $\rho = 0$ получается ближайшая аффинная функция вида $y = kx + b$, рассчитанная по методу наименьших квадратов. Синтаксис обращения: $\tilde{Y} = \text{csaps}(X, Y, \rho, \tilde{X})$, если требуется найти значения сплайна в заданном наборе точек $\tilde{X} = (\tilde{x}_{j,j=0}^m)$; или $P = \text{csaps}(X, Y, \rho)$, если требуется найти pp-форму аппроксимирующего сплайна.

Задание 2. Построить многочлен Лагранжа для узлов интерполяции $x_i = -1 + 0.25i$, $0 \leq i \leq 8$ и значений функции $y_i = x_i^2 + 0.1 \sin(c_n x_i)$, где $c_n = 5 + n$, n – номер варианта.

Задание 3. Построение кубического сплайна и оценка его погрешности. Для функции $f(x) = x^2 + 0.1 \sin(c_n x)$ по ее значениям на интервале $[-1, 1]$ с шагом δ построить сплайн $S_\delta(x)$ при помощи функции spline. Здесь $c_n = 50 + n$, а n – номер варианта. Сплайн зависит от δ и не совпадает с функцией f . Для диапазона значений δ от 0.1 до 0.14 с шагом 0.001 найти точность приближения: $J(\delta) = \max_{0 \leq x \leq 1} |f(x) - S_\delta(x)|$. Определить, при каком δ функция J достигает наибольшего значения и вывести на экран графики $f(x)$ и $S_\delta(x)$ при этом значении δ .

Задание 4. Величины x и y связаны зависимостью $y = x^2$, однако измерения этих величин произведены с погрешностями. Для значений аргумента x_i от -1 до 1 с шагом 0.1, т.е. при $x_i = -1 + 0.1i$, $0 \leq i \leq 20$ результаты измерений дали значения $y_i = x_i^2 + 0.1 \sin(c_n x_i)$, где $c_n = 20 + n$, n – номер варианта. Требуется при помощи функции interp1 построить сплайны методами: ближайшего соседа ('nearest'); ломаных ('linear'); кубической интерполяции дефекта 1 ('spline'); локальной кубической интерполяции дефекта 2 ('cubic'). При помощи функции csaps построить сглаживающий кубический сплайн с параметром $\rho = 0.995$. Вывести на график все синтезированные сплайны, а также исходную функцию $y = x^2$ на отрезке $[-1, 1]$. Для каждого сплайна $S(x)$ вычислить средний квадрат погрешности $S(x) - x^2$ на отрезке $[-1, 1]$.

Метод Рунге–Кутты решения задачи Коши для обыкновенных дифференциальных уравнений

Разработано много методов численного интегрирования дифференциальных уравнений [1–2, 4–9]. Остановимся на часто используемом методе Рунге–Кутты решения задачи Коши.

Рассмотрим следующую простейшую задачу Коши: на сегменте $[a, b]$ найти решение уравнения

$$\frac{dy}{dx} = f(x, y), \quad (1)$$

удовлетворяющее начальному условию $y(a) = y_0$, $y_0 \in \mathbb{R}$. Будем предполагать, что функция $f(x, y)$ достаточно гладкая.

Получим некоторые правила метода Рунге–Кутты. Возьмем промежуток $[x, x + h] \subset [a, b]$, обозначим $\Delta y = y(x + h) - y(x)$ и, используя формулу Тейлора, разложим Δy по степеням h :

$$\begin{aligned} \Delta y &= y'(x)h + \frac{y''(x)}{2!}h^2 + \dots + \frac{y^{(s)}(x)}{s!}h^s + \varrho, \\ \varrho &= Ah^{s+1}, \quad A = \frac{y^{(s+1)}(x + \theta h)}{(s+1)!}, \quad 0 \leq \theta \leq 1. \end{aligned}$$

Отметим, что $\varrho = O(h^{(s+1)})$.

Значение $y'(x) = f(x, y)$ определено уравнением (1). Дифференцируя обе части последнего равенства, находим

$$y''(x) = f'_x(x, y) + f'_y(x, y)y'(x) = f'_x(x, y) + f'_y(x, y)f(x, y),$$

$$y_j'''(x) = f_{xx}''(x, y) + 2f_{xy}''(x, y)f(x, y) + f_{yy}''(x, y)f^2(x, y) + f_y'(x, y)(f_x'(x, y) + f_y'(x, y)f(x, y)),$$

Таким образом,

$$\Delta y = hf(x, y) + \frac{h^2}{2}(f_x'(x, y) + f_y'(x, y)f(x, y)) + \frac{h^3}{6}(f_{xx}''(x, y) + 2f_{xy}''(x, y)f(x, y) + f_{yy}''(x, y)f^2(x, y) + f_y'(x, y)(f_x'(x, y) + f_y'(x, y)f(x, y))) + \dots \quad (2)$$

Введем три множества параметров:

$$\alpha_1, \alpha_2, \dots, \alpha_m;$$

$$\beta_{10},$$

$$\beta_{20}, \beta_{21},$$

$$\dots\dots\dots$$

$$\beta_{m0}, \beta_{m1}, \dots, \beta_{mm-1};$$

$$A_0, A_1, \dots, A_m.$$

Обозначим

$$k^{(0)} = hf(x, y),$$

$$k^{(1)} = hf(x + \alpha_1 h, y + \beta_{10} k^{(0)}),$$

$$k^{(2)} = hf(x + \alpha_2 h, y + \beta_{20} k^{(0)} + \beta_{21} k^{(1)}),$$

$$\dots\dots\dots$$

$$k^{(m)} = hf(x + \alpha_m h, y + \beta_{m0} k^{(0)} + \beta_{m1} k^{(1)} + \dots + \beta_{mm-1} k^{(m-1)})$$

и положим

$$\Delta y = \sum_{i=0}^m A_i k^{(i)}. \quad (3)$$

При выводе формул метода Рунге-Кутты разложение правой части равенства (3) по степеням h сравнивают с разложением (2) и выбирают значения введенных выше параметров так, чтобы в обоих разложениях совпадало максимальное количество первых членов с одинаковыми степенями h .

Так, при $m = 0$ равенства (3) и (2) имеют вид:

$$\Delta y = A_0 k^{(0)} = A_0 hf(x, y), \quad \Delta y = hf(x, y) + O(h^2).$$

Сравнив коэффициенты при hf , находим $A_0 = 1$, и мы приходим к правилу Рунге-Кутты первого порядка (методу Эйлера):

$$\Delta y \cong hf(x, y) \quad \text{с погрешностью метода } O(h^2).$$

(Погрешность метода порядка k равна $O(h^{k+1})$).

При $m = 1$ равенство (3) записывается следующим образом:

$$\begin{aligned} \Delta y &= A_0 k^{(0)} + A_1 k^{(1)} \\ &= A_0 hf(x, y) + A_1 hf(x + \alpha_1 h, y + \beta_{10} k^{(0)}) \\ &= A_0 hf(x, y) + A_1 h \left(f(x, y) + f_x'(x, y)\alpha_1 h + f_y'(x, y)\beta_{10} k^{(0)} + O(h^2) \right) \\ &= A_0 hf(x, y) + A_1 h (f(x, y) + f_x'(x, y)\alpha_1 h + f_y'(x, y)\beta_{10} hf(x, y) + O(h^2)) \\ &= (A_0 + A_1)hf(x, y) + A_1 h^2 (\alpha_1 f_x'(x, y) + \beta_{10} f_y'(x, y)f(x, y) + O(h^3)), \end{aligned}$$

а разложение (2) —

$$\Delta y = hf(x, y) + \frac{1}{2} h^2 (f_x'(x, y) + f_y'(x, y)f(x, y)) + O(h^3).$$

Приравняв коэффициенты при hf , $h^2 f_x'$, $h^2 f_y' f$, приходим к системе уравнений

$$\begin{cases} A_0 + A_1 = 1, \\ A_1 \alpha_1 = 0.5, \\ A_1 \beta_{10} = 0.5, \end{cases}$$

одним из решений которой является

$$\begin{cases} A_0 = A_1 = 0.5, \\ \alpha_1 = \beta_{10} = 1, \end{cases}$$

и получается широко известное правило Рунге-Кутты второго порядка ("предиктор - корректор")

$$\Delta y \cong \frac{h}{2} (f(x, y) + f(x + h, y + hf(x, y))) \quad (4)$$

Замечание. Равенству (4) можно придать следующую форму:

$$\Delta y \cong \frac{h}{2} (f(x, y) + f(x + h, y(x + h))),$$

что является аналогом формулы трапеций для интеграла

$$\Delta y = \int_x^{x+h} f(s, y(s)) ds. \quad (5)$$

При $A_1 = 1$ мы приходим еще к одному часто используемому правилу Рунге–Кутты второго порядка:

$$\Delta y \cong h(f(x + \frac{h}{2}, y + \frac{h}{2}f(x, y)). \quad (6)$$

Замечание. Это правило — аналог квадратурной формулы средних прямоугольников для интеграла (5).

Действуя аналогично предыдущему, получают различные правила Рунге–Кутты третьего, четвертого, пятого и более высоких порядков.

При использовании метода Рунге–Кутты разбивают сегмент $[a, b]$ на n равных частей точками

$$x_j = a + hj, \quad h = \frac{b-a}{n}, \quad j = 0, 1, 2, \dots, n,$$

к каждому сегменту $[x_j, x_{j+1}]$ последовательно применяют то или иное правило Рунге–Кутты.

Обозначим $y_j = y(x_j)$, $\Delta y_j = y_{j+1} - y_j$. Правилу (4) тогда будет соответствовать схема:

$$\begin{cases} \tilde{y}_0 &= y_0, \\ k_j^{(0)} &= hf(x_j, \tilde{y}_j), \\ k_j^{(1)} &= hf\left(x_j + h, \tilde{y}_j + k_j^{(0)}\right), \\ \Delta \tilde{y}_j &= \frac{1}{2} \left(k_j^{(0)} + k_j^{(1)}\right), \\ \tilde{y}_{j+1} &= \tilde{y}_j + \Delta \tilde{y}_j, \quad j = 0, 1, 2, \dots, n-1. \end{cases} \quad (7)$$

Схема широко используемого правила Рунге–Кутты четвертого порядка имеет вид:

$$\begin{cases} \tilde{y}_0 &= y_0, \\ k_j^{(0)} &= hf(x_j, \tilde{y}_j), \\ k_j^{(1)} &= hf\left(x_j + \frac{h}{2}, \tilde{y}_j + \frac{1}{2}k_j^{(0)}\right), \\ k_j^{(2)} &= hf\left(x_j + \frac{h}{2}, \tilde{y}_j + \frac{1}{2}k_j^{(1)}\right), \\ k_j^{(3)} &= hf\left(x_j + h, \tilde{y}_j + k_j^{(2)}\right), \\ \Delta \tilde{y}_j &= \frac{1}{6} \left(k_j^{(0)} + 2k_j^{(1)} + 2k_j^{(2)} + k_j^{(3)}\right), \\ \tilde{y}_{j+1} &= \tilde{y}_j + \Delta \tilde{y}_j, \quad j = 0, 1, 2, \dots, n-1. \end{cases} \quad (8)$$

Замечание. Это правило — аналог квадратурной формулы Симпсона для интеграла (5).

Замечание. Методами Рунге–Кутты, Эйлера, Адамса и другими можно пользоваться при решении задачи Коши как для одного уравнения, так и для системы обыкновенных дифференциальных уравнений. При этом для каждого значения j вычисляются приращения всех неизвестных функций. В качестве примера приведем схему (7) Рунге–Кутты для задачи Коши в случае нормальной системы обыкновенных дифференциальных уравнений:

$$\frac{dy_s}{dx} = f_s(x, y_1, y_2, \dots, y_m), \quad y_s(a) = y_{s0},$$

где $y_{s0} \in \mathbf{R}$, $s = 1, 2, \dots, m$, $x \in [a, b]$. Неизвестные функции $y_s = y_s(x)$.

Схема (7) в этом случае имеет вид:

$$\begin{cases} \tilde{y}_{s0} &= y_{s0}, \\ k_{sj}^{(0)} &= hf_s(x_j, \tilde{y}_{1j}, \tilde{y}_{2j}, \dots, \tilde{y}_{mj}), \\ k_{sj}^{(1)} &= hf_s\left(x_j + h, \tilde{y}_{1j} + k_{1j}^{(0)}, \tilde{y}_{2j} + k_{2j}^{(0)}, \dots, \tilde{y}_{mj} + k_{mj}^{(0)}\right), \\ \Delta \tilde{y}_{sj} &= \frac{1}{2} \left(k_{sj}^{(0)} + k_{sj}^{(1)}\right), \\ \tilde{y}_{s,j+1} &= \tilde{y}_{sj} + \Delta \tilde{y}_{sj}, \quad \begin{matrix} j = 0, 1, 2, \dots, n-1, \\ s = 1, 2, \dots, m. \end{matrix} \end{cases}$$

Здесь обозначено $h = (b-a)/n$, $x_j = a + hj$, $y_{sj} = y_s(x_j)$.

Системы обыкновенных дифференциальных уравнений другого вида (а также одно дифференциальное уравнение выше первого порядка) с помощью введения новых неизвестных функций часто удается свести к нормальной системе. Например, задачу Коши

$$y''(x) + xy'(x) + 2y(x) = 0,$$

$$y(0) = 1, \quad y'(0) = 2$$

нетрудно привести к соответствующей задаче Коши для системы двух обыкновенных дифференциальных уравнений с двумя неизвестными функциями $y(x)$ и $u(x)$:

$$\begin{cases} y'(x) &= u(x), \\ u'(x) &= -2y(x) - xu(x), \end{cases} \quad \begin{matrix} y(0) = 1, \\ u(0) = 2, \end{matrix}$$

если положить $u(x) = y'(x)$.

Замечание. Простейшим контролем правильности выбора шага h интегрирования является пересчет решения с шагом $2h$ на сегменте $[a, b]$. При этом принято считать, что если совпадают первые s значащих цифр решений, найденных с шагом h и $2h$, то эти цифры верные.

Примечание. Имеются компьютерные программы, которые реализуют схемы различного порядка метода Рунге-Кутты. Прежде, чем использовать ту или иную программу, следует внимательно ознакомиться с инструкцией к ней, а также с комментариями в тексте программы. Рекомендуется, если это возможно, предварительно решить какую-нибудь аналогичную задачу, имеющую точное решение, — "модельную задачу" и сравнить результаты.

В системе MATLAB, версия 4.0, имеются программы, которые используют правила Рунге-Кутты второго и третьего порядка (функция ode23) и четвертого и пятого порядка (функция ode45) с автоматическим выбором шага интегрирования (соответствующие им файлы (ode23.m, ode23.c и ode45.m) находятся в директории \MATLAB\TOOLBOX\MATLAB\FUNFUN). Демонстрация работы этих программ осуществляется запуском с командной строки MATLAB программы demo, а затем выбором в окне Numerical Analysis строки ODEs. При этом на экран монитора будут выводиться соответствующие команды, комментарии, результаты счета, графики решений. Комментарии к программам можно получить, нажав кнопку Help в командном окне MATLAB, в появившейся табличке – строку Table of Contents ..., в возникшем окне – FUNFUN, затем – имя нужной программы (или обратиться к указанным выше файлам).

В более поздних версиях MATLAB содержатся и другие программы решения задач для обыкновенных дифференциальных уравнений, например, ode113, ode15s, ode23s.

Задание 5. Используя функции ode23 и ode45, найти на промежутке изменения t $[0; 18]$ решение задачи Коши для системы уравнений Лотке-Вольтерра

$$\begin{cases} y_1'(t) = y_1(t) - \alpha y_1(t)y_2(t), \\ y_2'(t) = -y_2(t) + \beta y_1(t)y_2(t) \end{cases}$$

с заданными начальными условиями и значениями $\alpha, \beta \in \mathbb{R}$, с помощью функции plot вывести на экран графики решения.

№	α	β	$y_1(0)$	$y_2(0)$
1.	0.1	0.2	25	26
2.	"	"	"	20
3.	0.2	"	"	"
4.	"	"	30	"
5.	"	0.1	"	"
6.	0.2	0.15	30	20
7.	"	"	40	"
8.	0.3	"	"	"
9.	"	"	25	"
10.	"	"	"	35
11.	"	0.15	"	30
12.	0.25	"	"	"
13.	"	"	30	"
14.	"	"	"	25
15.	"	0.28	"	"

№	α	β	$y_1(0)$	$y_2(0)$
16.	0.17	0.21	30	25
17.	"	"	15	"
18.	"	"	"	19
19.	"	"	20	"
20.	0.12	"	"	"
21.	0.25	"	"	"
22.	"	0.31	"	"
23.	"	0.17	"	"
24.	"	"	"	31
25.	"	"	"	23
26.	"	"	"	"
27.	0.16	"	"	"
28.	"	"	"	25
29.	"	0.23	"	"
30.	0.13	0.15	27	28

Задание 6. Используя те же функции, что и в задании 5, найти на промежутке $[0; 8.1]$ решение задачи Коши для уравнения

$$\frac{d^3 y(x)}{dx^3} + 2x \frac{d^2 y(x)}{dx^2} + 15 \cos 5x \frac{dy(x)}{dx} - \sqrt{x} y(x) = \alpha x e^{-\beta x} \sin(\gamma x)$$

и вывести на экран монитора графики решения.

№	α	β	γ	$y(0)$	$y'(0)$	$y''(0)$
1.	4.1	2.3	12.6	1.23	8.5	-5.7
2.	"	"	"	"	-8.21	"
3.	"	"	"	-9.3	"	"
4.	"	"	"	"	"	6.81
5.	"	"	"	4.72	"	"
6.	4.1	2.3	12.6	1.23	8.5	-4.72
7.	"	"	8.32	"	"	"
8.	"	"	"	"	"	-8.41
9.	"	"	"	-6.48	"	"
10.	"	"	"	"	7.32	"
11.	"	"	"	"	-3.58	"
12.	"	"	"	5.41	"	"
13.	"	-0.21	"	"	"	"
14.	-18.2	"	"	"	"	"
15.	"	-0.47	"	"	"	"

№	α	β	γ	$y(0)$	$y'(0)$	$y''(0)$
16.	"	0.53	"	"	"	"
17.	"	"	-9.74	"	"	"
18.	"	"	"	9.63	"	"
19.	"	"	"	"	8.76	"
20.	"	"	"	"	"	-20.4
21.	16.4	"	"	"	"	"
22.	"	0.15	"	"	"	"
23.	"	0.65	"	"	"	"
24.	"	"	18.3	"	"	"
25.	"	"	9.21	"	"	"
26.	3.71	0.65	"	"	"	"
27.	"	"	"	"	-4.76	"
28.	"	"	"	13.2	"	"
29.	"	"	"	"	-10.31	-12.7
30.	"	"	"	"	"	-20.82

ЛИТЕРАТУРА

1. Батвалов Н.С. Численные методы. Т. 1. — М., Наука: 1973, 632 с.
2. Березин И.С., Жидков Н.П. Методы вычислений, Т. 2. — М., Наука: 1962, 620 с.
3. Дьяконов В.П. Справочник по применению системы РС MatLAB. — М., Наука: 1993, 112 с.
4. Коллатц Л. Численные методы решения задач дифференциальных уравнений. — М., ИЛ: 1953, 460 с.
5. Крылов В.И., Бобков В.А., Монастырный П.И. Вычислительные методы. Т. 2. — М., Наука: 1977, 400 с.
6. Ланс Д.Н. Численные методы для быстродействующих вычислительных машин. — М., Наука: 1962, 208 с.
7. Милн В.Э. Численное решение дифференциальных уравнений. — М., ИЛ: 1955, 292 с.
8. Мысовский И.П. Лекции по методам вычислений. — СПб.: Издательство С.-Петербургского университета, 1998, 472с.
9. Форсайт Дж., Малькольм М., Моулер К. Машинные методы математических вычислений. — М., Мир: 1980, 279 с.

ЛР № 040815 от 22.05.97.

Подписано к печати 13.07.2000 г. Формат бумаги 60X90 1/16. Бумага офсетная.

Печать ризографическая. Объем 1.6 п.л. Тираж 150 экз. Заказ 1471.

Отпечатано в отделе оперативной полиграфии НИИХ СПбГУ
с оригинал-макета заказчика.

198904, Санкт-Петербург, Старый Петергоф, Университетский пр. 2.